

Async Programming in Python: Fundamentals

파이썬 비동기 프로그래밍 기초

비동기 입문: 개념과 문법

동기와 비동기



3개의 외부 API 요청을 순서대로 보낸다면

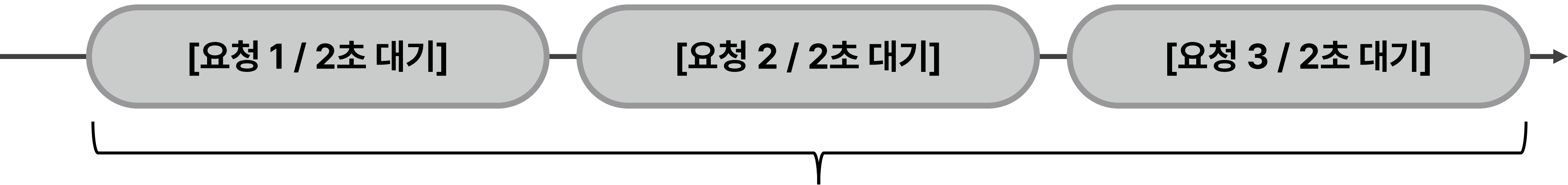
[요청 1 / 2초 대기]

[요청 2 / 2초 대기]

[요청 3 / 2초 대기]



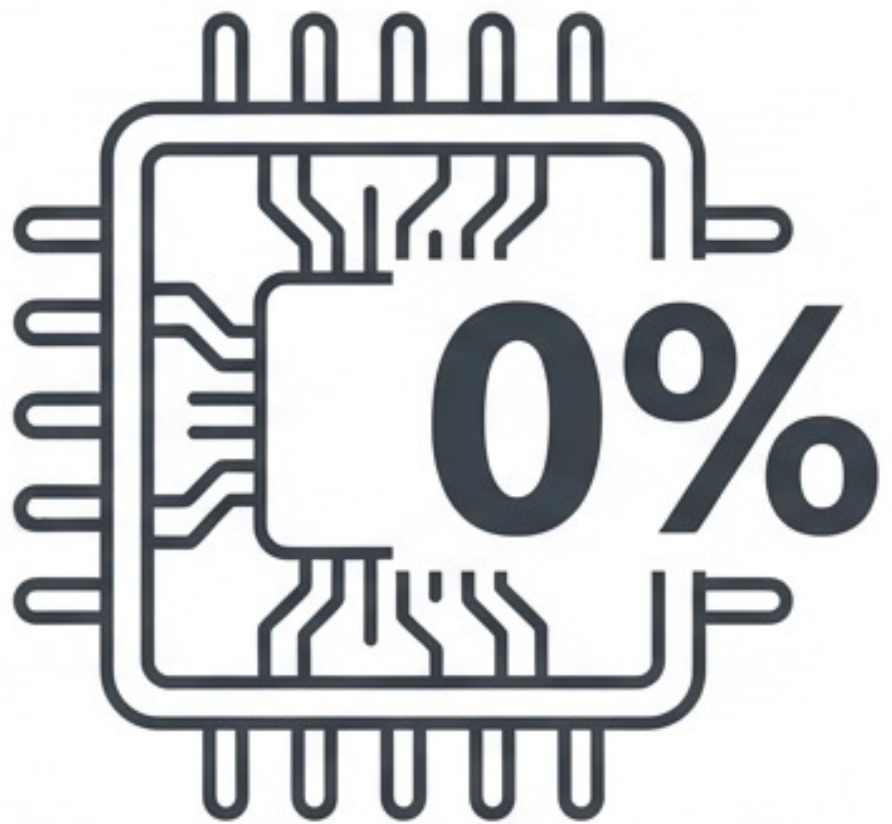
기다림의 시간은 계속 누적됩니다



$$2\text{초} + 2\text{초} + 2\text{초} = \text{총 } 6\text{초 소요}$$

현재 작업이 끝날 때까지 다음 작업은 시작할 수 없음

이 시간 동안 프로그램은 무엇을 할까?



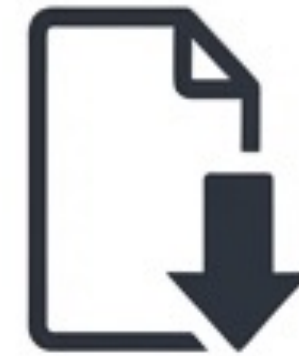
프로그램은 CPU가 열심히 계산을 하는 것이 아니라,
그저 외부 응답이 돌아오기를 기다리며 멈춰 있는 상태에 가까움



멈춰서 기다리는 상황은 어디서나 발생함



여러 개의 외부 API 동시 호출



파일 읽기 및 다운로드



데이터베이스 조회 결과 대기



네트워크 응답 대기



비동기 프로그래밍

(Asynchronous Programming)

하나의 작업이 응답을 기다리는 동안, 다른 작업을 먼저 진행



단일 차선의 한계 : 동기 실행

작업 1

실행

대기 - 아무 일도 하지 않음

작업 2

실행

대기 - 아무 일도 하지 않음

작업 3

실행

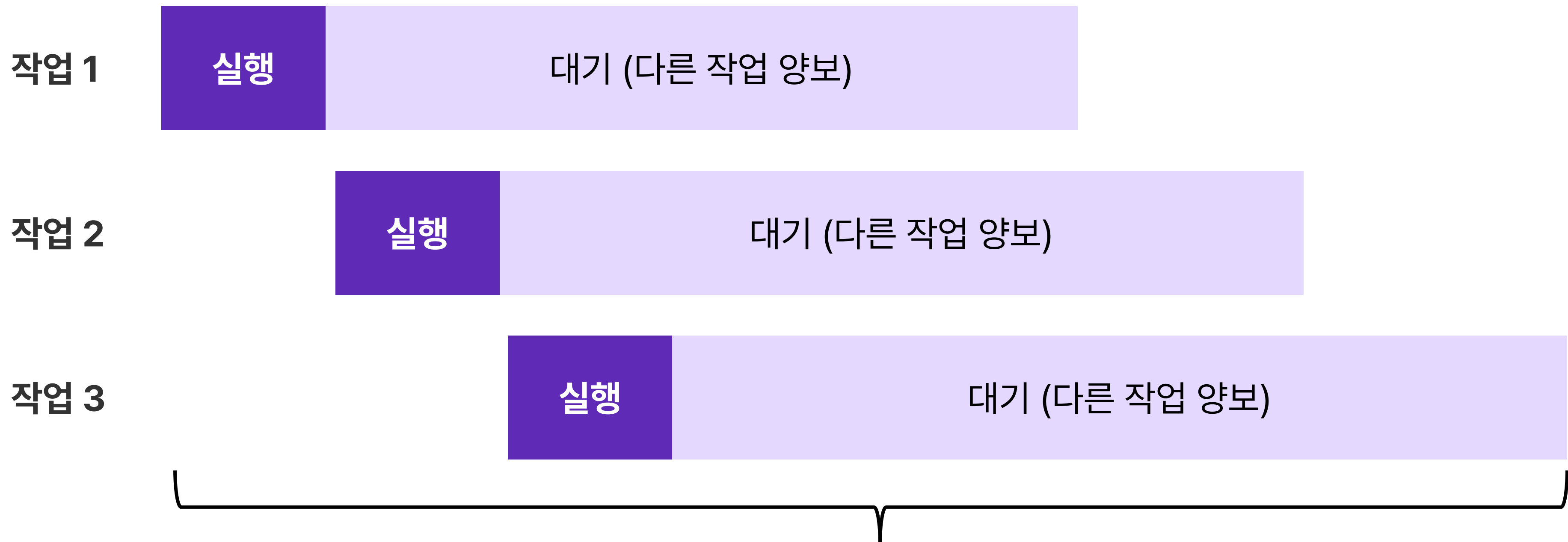
대기 - 아무 일도 하지 않음

총 소요 시간 = 작업 1 + 작업 2 + 작업 3

일반적인 프로그램은 위에서 아래로 순차적으로 실행됨
기다리는 시간이 긴 작업이 섞이면 전체 실행 시간이 걸잡을 수 없이 길어짐



대기 시간의 중첩: 비동기 실행



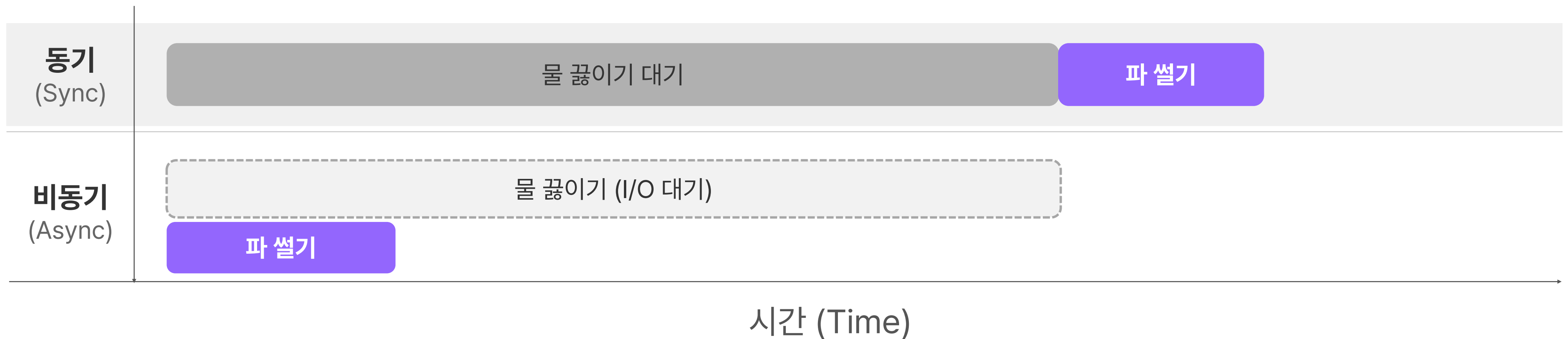
총 소요 시간 = 가장 긴 대기 시간과 유사

하나의 작업이 응답을 기다리는 동안, 멈춰 있지 않고 다른 작업을 먼저 진행함
이것이 비동기의 핵심



비동기의 본질 : 라면 물이 끓을 때까지 냄비만 봐야 할까?

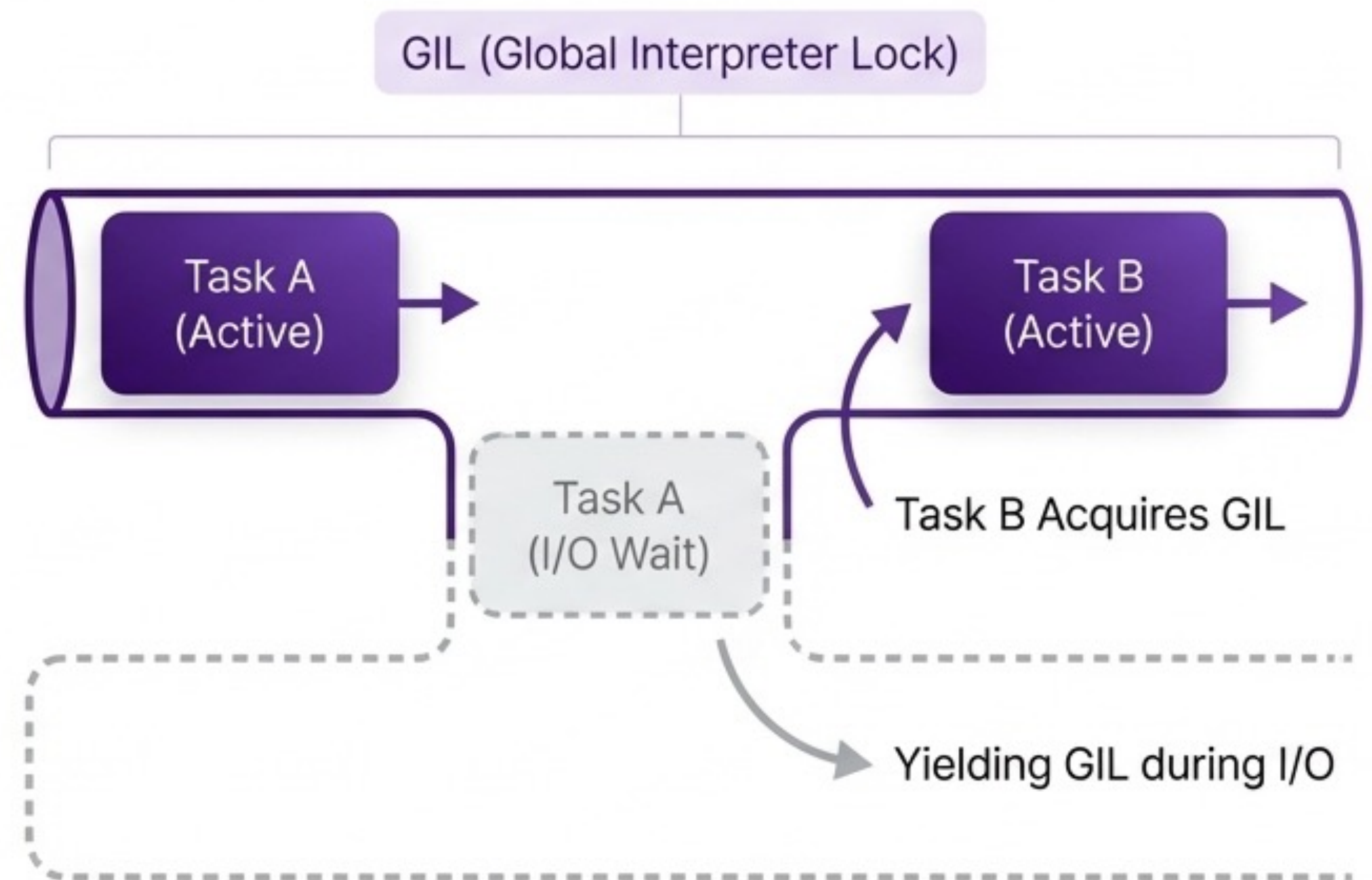
비동기 프로그래밍은 '노는 시간(I/O 대기)'을 다른 작업에 양보하여
전체 실행 시간을 극적으로 단축하는 기술



파이썬의 태생적 제약(GIL)을 우회하는 방법

파이썬은 **GIL(Global Interpreter Lock)**로 인해 한 번에 하나의 스레드만 파이썬 바이트코드를 실행할 수 있음

따라서 스레드를 늘려도 '연산' 속도는 빨라지지 않지만, 비동기 처리를 통해 '대기 시간'을 다른 작업에 할당하면 하나의 프로세스만으로도 엄청난 효율을 냄





동기와 비동기

프로세스 : 실행 중인 프로그램의 인스턴스

- 독립적인 메모리 공간 보유
- 다른 프로세스와 격리되어 동작

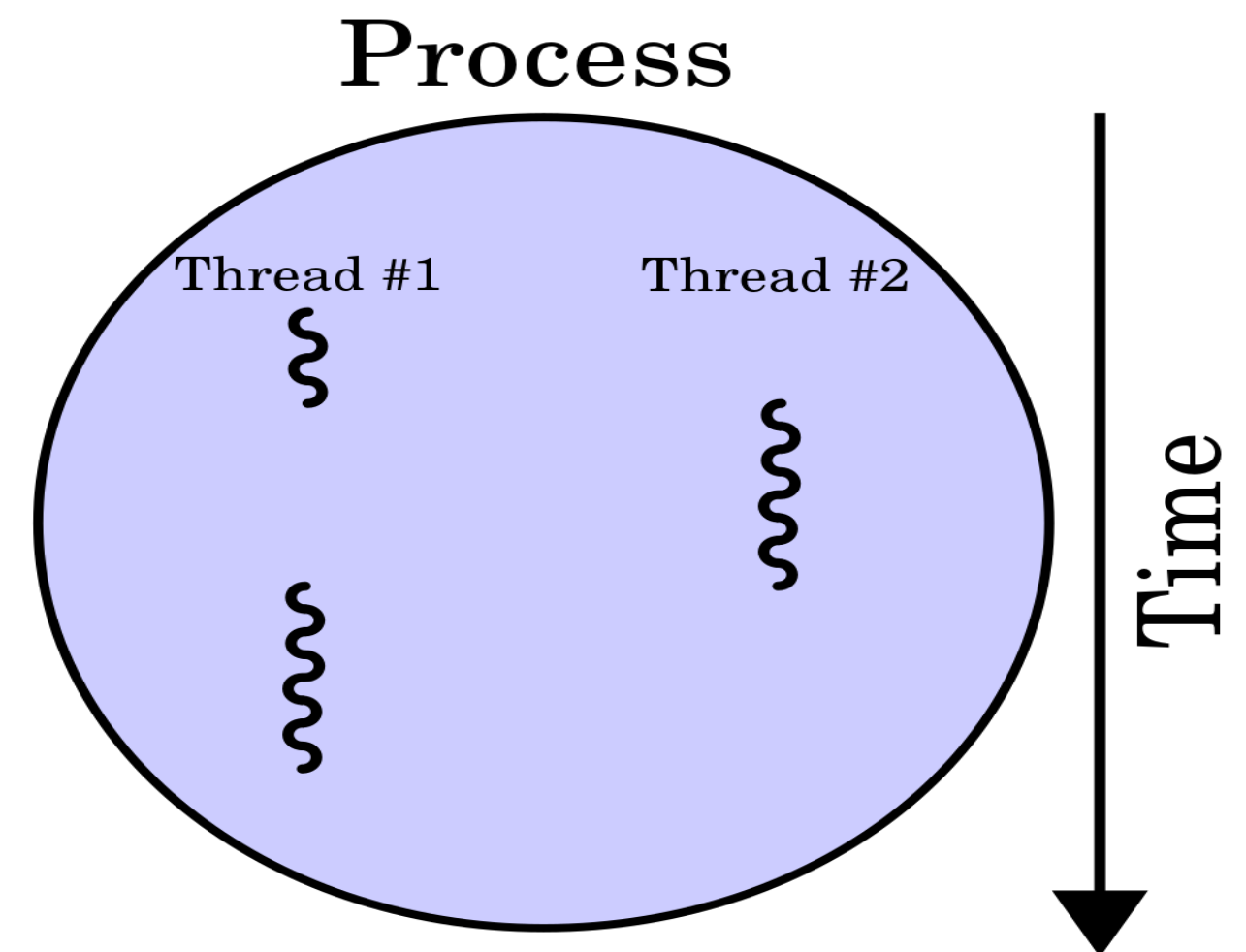
```
CPU[11] 2.0% Tasks: 16 total, 1 running
Mem[1111111111111111] 13/123MB Load average: 0.37 0.12 0.04
Swap[ ] 0/109MB Uptime: 00:00:50
```

PID	USER	PR	NI	UIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
3692	per	15	0	2424	1204	980	R	2.0	1.0	0:00.24	htop
1	root	16	0	2952	1852	532	S	0.0	1.5	0:00.77	/sbin/init
2236	root	20	-4	2316	728	472	S	0.0	0.6	0:01.06	/sbin/udevd --daemon
3224	dhcp	18	-2	2412	552	244	S	0.0	0.4	0:00.00	dhclient3 -e IF_M
3488	root	18	0	1692	516	448	S	0.0	0.4	0:00.00	/sbin/getty 38400
3491	root	18	0	1696	520	448	S	0.0	0.4	0:00.01	/sbin/getty 38400
3497	root	18	0	1696	516	448	S	0.0	0.4	0:00.00	/sbin/getty 38400
3500	root	18	0	1692	516	448	S	0.0	0.4	0:00.00	/sbin/getty 38400
3501	root	16	0	2772	1196	936	S	0.0	0.9	0:00.04	/bin/login --
3504	root	18	0	1696	516	448	S	0.0	0.4	0:00.00	/sbin/getty 38400
3539	syslog	15	0	1916	704	564	S	0.0	0.6	0:00.12	/sbin/syslogd -u s
3561	root	18	0	1840	536	444	S	0.0	0.4	0:00.79	/bin/dd bs 1 if /p
3563	klog	18	0	2472	1376	408	S	0.0	1.1	0:00.37	/sbin/klogd -P /va
3590	daemon	25	0	1960	428	308	S	0.0	0.3	0:00.00	/usr/sbin/atd
3604	root	18	0	2336	792	632	S	0.0	0.6	0:00.00	/usr/sbin/cron
3645	per	15	0	5524	2924	1428	S	0.0	2.3	0:00.45	-bash

F1Help F2Setup F3SearchF4InvertF5Tree F6SortByF7Nice -F8Nice -F9Kill F10Quit

스레드 : 프로세스 내 실행 작업의 단위

- 하나의 프로세스 = 하나 이상의 스레드
- 같은 프로세스의 메모리를 공유

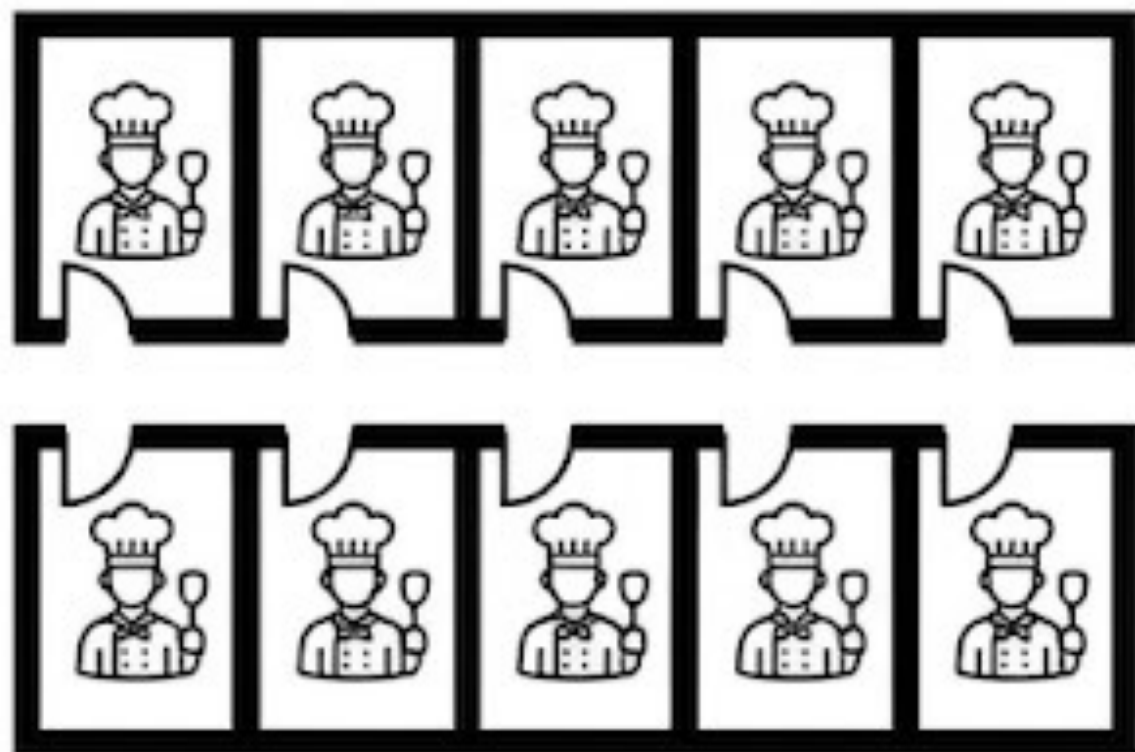


출처: [ko.wikipedia.org/wiki/스레드_\(컴퓨팅\)](https://ko.wikipedia.org/wiki/스레드_(컴퓨팅))



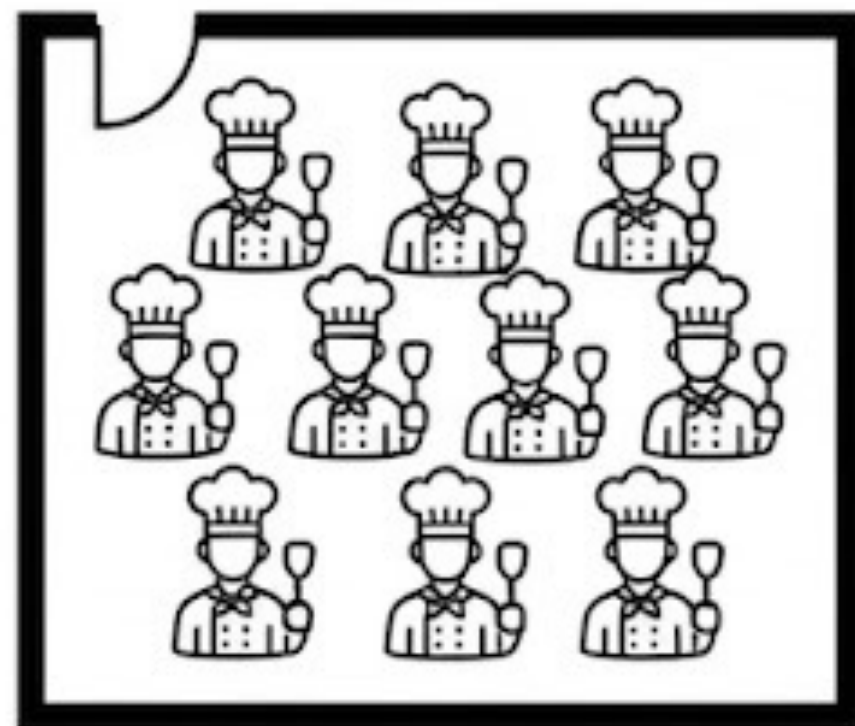
동시성 엔진 아키텍처 비교

멀티프로세싱 (Multiprocessing)



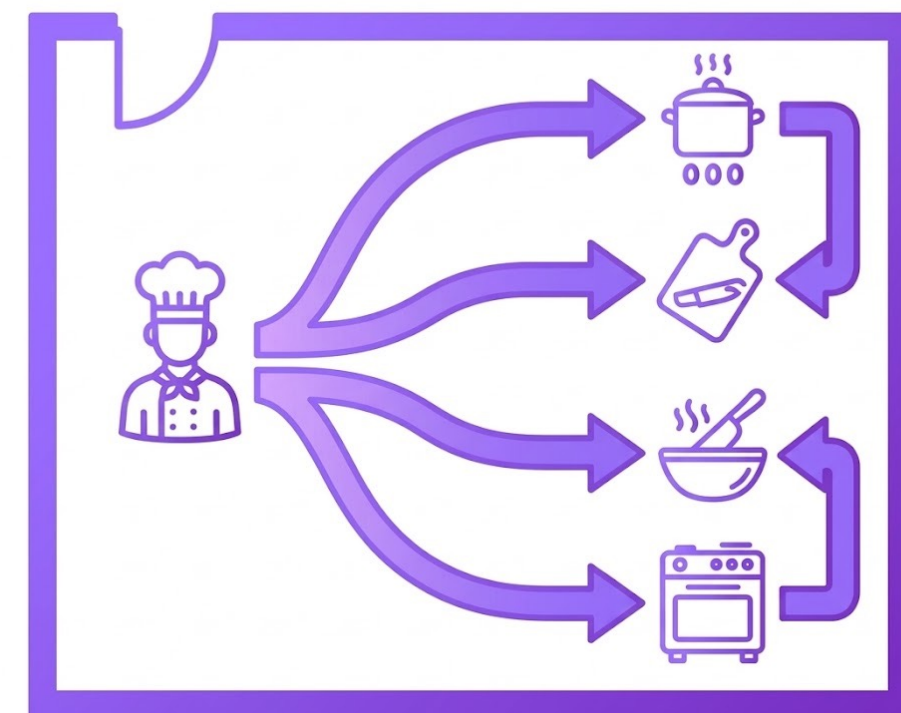
10개의 주방, 10명의 셰프
독립 메모리, 높은 CPU 활용
문맥 교환 오버헤드 큼

스레딩 (Threading)



1개의 주방, 10명의 셰프
메모리 공유,
OS 주도 선점형 스케줄링
GIL 제약 존재

비동기 (AsyncIO)

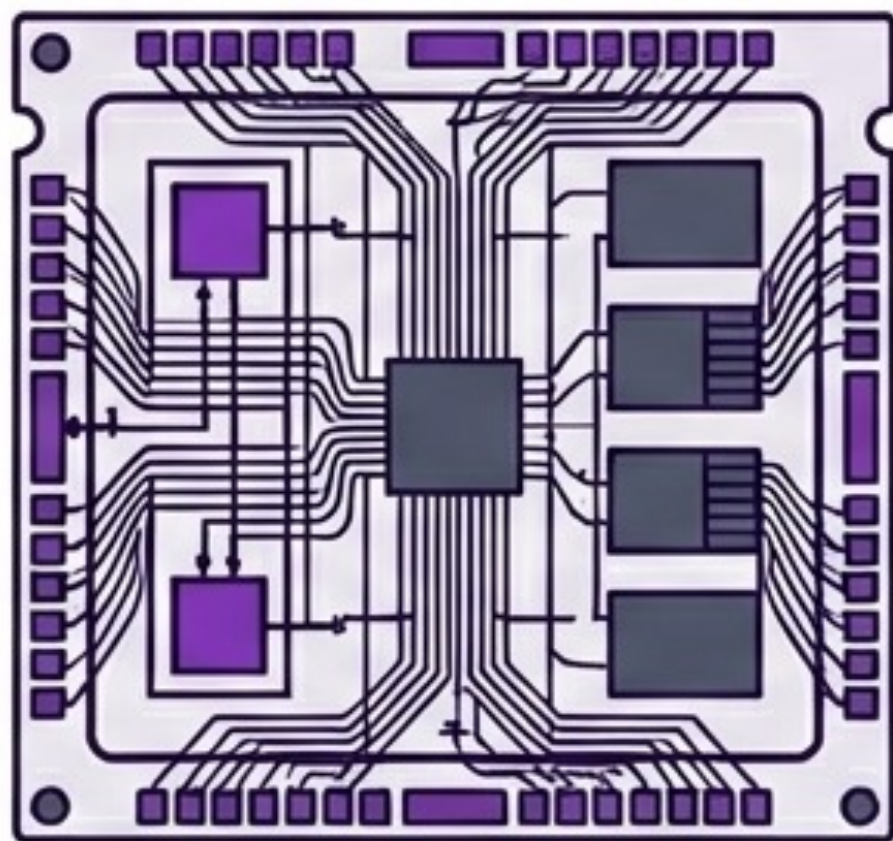


1개의 주방, 1명의 영리한 셰프
단일 프로세스/스레드
협력적 스케줄링
I/O 대기 최소화



병목의 근원 : CPU 연산 vs. I/O 대기

CPU-Bound (연산 병목)



복잡한 수학 연산
클럭 속도 향상이나 멀티프로세싱 필요

I/O-Bound (대기 병목)

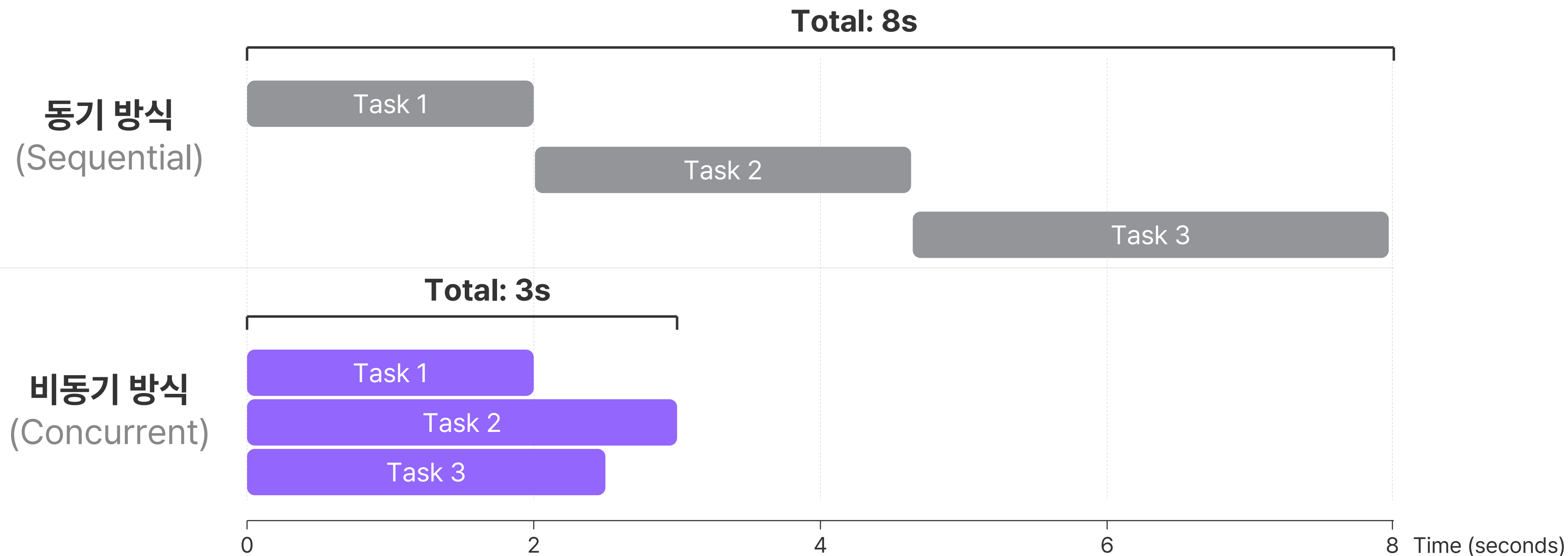


DB 조회, 외부 API 응답 대기
현대 웹 서버의 주된 병목

비동기(Async)는 바로 이 I/O 대기 시간의 낭비를 없애는 기술

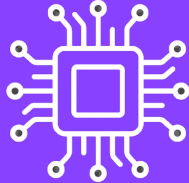
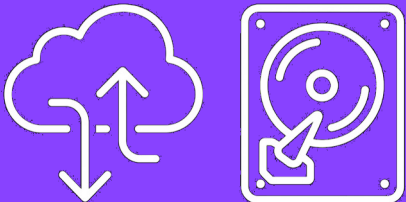


I/O 병목 해결



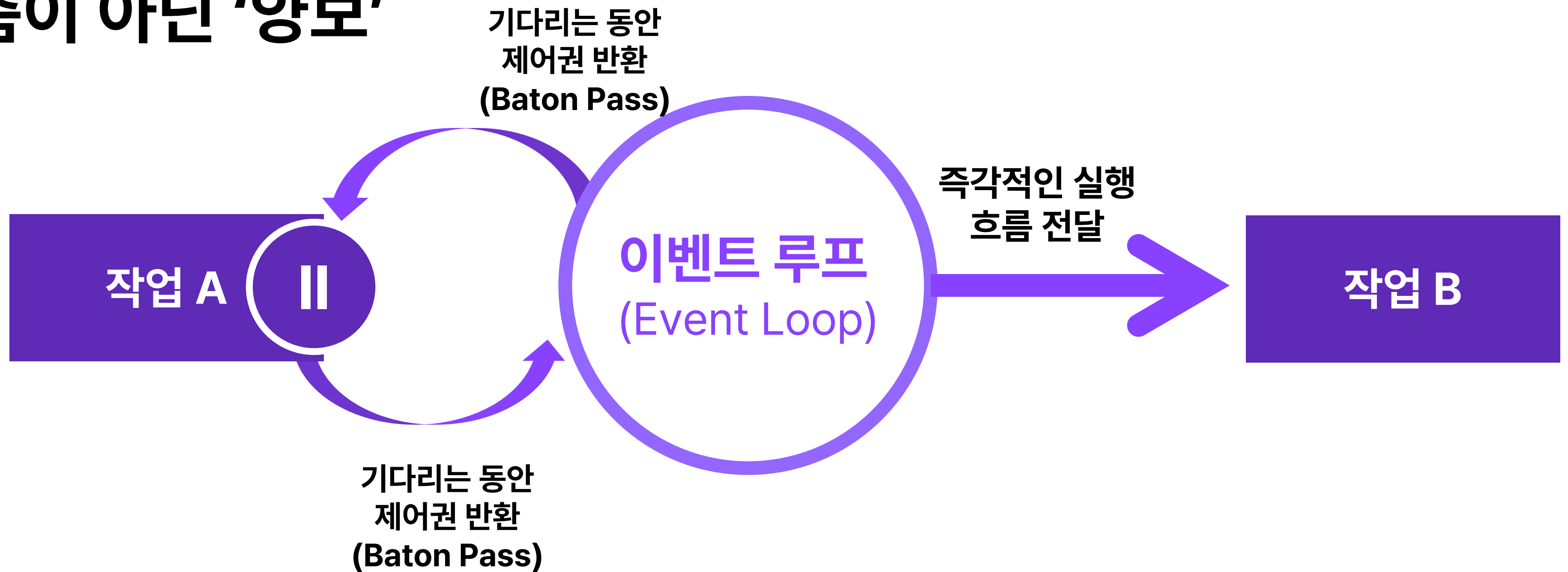
하나의 요청이 끝나기를 기다리며 멈춰있지 않고 겹치는 시간만큼 전체 실행 시간이 극적으로 단축됨



구분	CPU 바운드 	I/O 바운드 
특징	수치 계산, 이미지 처리 등 연산 속도가 핵심	네트워크 요청, DB 쿼리, 파일 읽기 등 대기 시간이 핵심
병목 원인	프로세서의 물리적 연산 한계	외부 응답을 기다리며 멈춰있는 시간 (노는 시간)
최적의 해결책	멀티프로세싱 (Multiprocessing)	비동기 프로그래밍 (Async/Await)



멈춤이 아닌 '양보'



비동기는 동시에 여러 연산을 하는 것이 아니라, 대기 상태에 빠진 작업이
다른 작업에게 '실행 흐름'을 즉각적으로 양보하는 구조

Python에서의 비동기 프로그래밍



비동기의 도구 상자 : asyncio 패키지

```
import asyncio  
# 파이썬 표준 라이브러리 - 별도 설치 불필요
```

asyncio란?

- 파이썬의 **비동기 프로그래밍 전용 패키지**
- 이벤트 루프 + 문법 + 유틸리티를 **한 곳에**

async def
비동기 함수
정의

await
결과 기다림
(양보)

asyncio.run()
이벤트 루프
시작

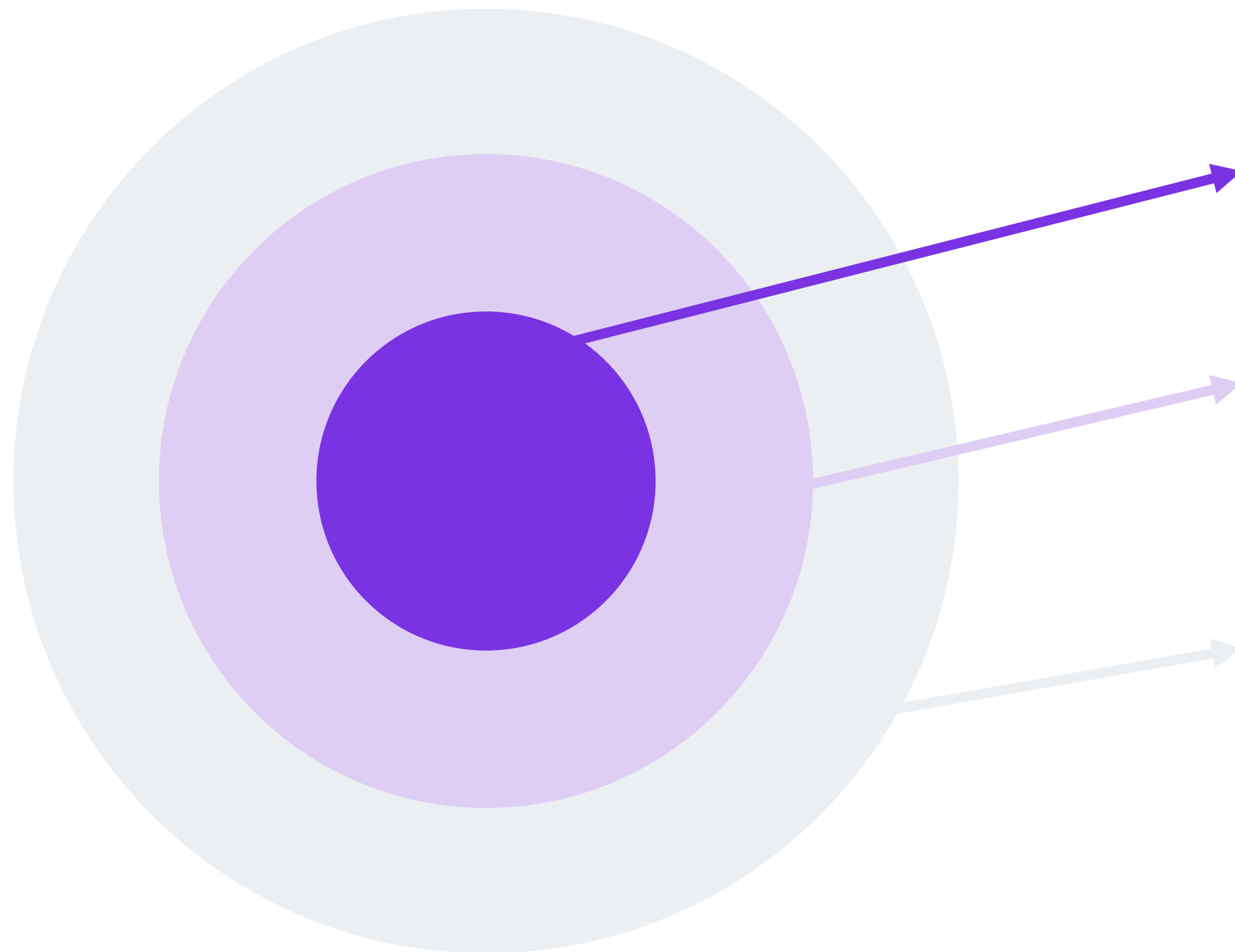
create_task()
태스크 등록
(백그라운드)

gather()
여러 작업
동시 수집

sleep()
비동기 대기
(양보)



비동기 세계를 지탱하는 3가지 핵심 계층



이 계층적 구조(코루틴 → 태스크 → 루프)를 이해하는 것이 비동기 설계의 첫걸음

코루틴 (Coroutine)

비동기로 동작하는 함수 그 자체 (가장 작은 단위)

```
async def
```

태스크 (Task)

루프에 예약되어 실행 대기 중인 작업

```
create_task
```

루프 (Event Loop)

모든 비동기 작업을 지휘하는 중앙 엔진

```
asyncio.run
```




비동기 함수의 탄생: `async def`

```
def hello_sync():  
    return 'sync hello'
```



즉시 결과 반환

```
async def hello_async():  
    return 'async hello'
```

Coroutine 객체 생성

`async def`로 정의된 함수는 호출 시 바로 실행되지 않음
대신 비동기적으로 실행될 수 있는 상태인 코루틴(coroutine)을 반환



비동기 함수 만들기 : `async def`

```
# 1. 기본 정의
async def fetch(url: str) -> dict:
    await asyncio.sleep(1)
    return {"url": url}

# 2. 호출하면 코루틴 객체 반환
coro = fetch("http://...")
print(type(coro))
# <class 'coroutine'>

# 3. 타입 힌트 · 매개변수 · return 모두 일반 함수와
동일
# 차이점은 단 하나: await를 쓸 수 있다
```

```
async def 함수명(매개변수):
    ...
```

핵심 포인트

- `async def` = `def`에 `async`만 붙인 것
- 매개변수 · 타입 힌트 · `return` 모두 **일반 함수와 동일**
- 호출하면 **바로 실행 안 됨** → 코루틴 객체 반환
- 이 함수를 코루틴 함수라고 부름



def vs async def : 호출 결과의 차이

```
# 일반 함수
def greet():
    return "hello"

result = greet()
print(result)
# "hello" ← 값이 바로 반환!
```

```
# 비동기 함수
async def greet():
    return "hello"

result = greet()
print(type(result))
# <class 'coroutine'> ← 값이 아님!
```

핵심 차이

- def 호출 → 함수 본문이 즉시 실행되고 결과값 반환
- async def 호출 → 본문 실행 없이 코루틴 객체만 반환
- 그냥 호출하면 **RuntimeWarning** 발생!



기다림의 문법 : await



- ✓ '작업이 끝날 때까지 기다리되, 다른 작업이 있으면 먼저 하라'는 의미
- ✓ 반드시 `async def`로 정의된 비동기 함수 내부에서만 사용 가능
- ✓ 아무 객체나 기다릴 수는 없고, 기다릴 수 있는 대상 (비동기 처리가 가능한 대상, Awaitable 객체) 앞에만 붙일 수 있음



비동기 실행 대기 : await

```
async def fetch_data():
    print("요청 시작")
    await asyncio.sleep(2) # 여기서 양보!
    print("응답 수신")
    return "data"

async def main():
    result = await fetch_data()
    print(result) # "data"

asyncio.run(main())
```

결과 = await awaitable 객체

핵심 포인트

- 코루틴의 **결과를 기다림** (반환값 수신)
- 기다리는 동안 **제어권을 이벤트 루프에 양보**
- **async def** 함수 안에서만 사용 가능
- 일반 함수에서 사용시 SyntaxError 발생



비동기의 핵심 동작 : `asyncio.sleep()`

```
import asyncio

async def task_a():
    print("A: 시작")
    await asyncio.sleep(2)  # 양보!
    print("A: 완료")

async def task_b():
    print("B: 시작")
    await asyncio.sleep(1)  # 양보!
    print("B: 완료")
```

`await asyncio.sleep(초)`

인자: 대기 시간 (**초 단위**, float/int)

반환값: **None**

조건: **async def** 안에서만 사용 (await 필수)

핵심 포인트

- 지정한 시간 동안 제어권을 이벤트 루프에 양보하는 **비동기 함수**
- 반드시 **await**와 함께 사용 (코루틴 함수이므로)
- 나만 쉬고, 다른 태스크가 **그 틈에 실행**
- 실제 I/O 대기를 **시뮬레이션**하는 용도



비동기 함수의 호출 방식

```
async def main():  
    print("Hello")  
  
# 일반적인 호출 시도  
main()
```



일반적인 호출 : 실행 불가

main() 직접 호출 → 코루틴 객체만 반환, 실행되지 않음
이벤트 루프가 없으면 코루틴은 **설계도일 뿐** →
asyncio.run()이 시동 키

```
import asyncio  
  
async def main():  
    print("Hello")  
  
# 올바른 진입점  
asyncio.run(main())
```

**Async Event
Loop Zone**

이벤트 루프 진입 성공



문법 정리 : asyncio.run()

```
# 기본 사용법
asyncio.run( main() )

# 전체 예제
import asyncio

async def main():
    print("start")
    await asyncio.sleep(1)
    print("done")

asyncio.run(main())
# 출력: start → (1초 후) → done
```

asyncio.run(코루틴)

인자: **코루틴 객체** (async def의 호출 결과)

반환값: 코루틴의 **실행 결과값**

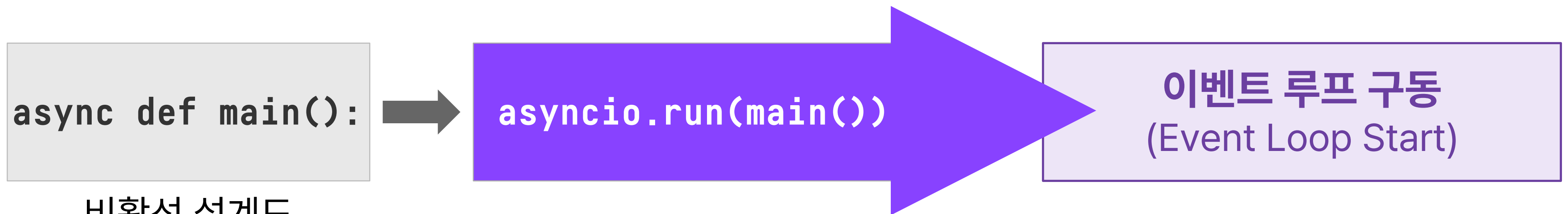
조건: **async 함수 밖**에서 호출 (프로그램 진입점)

핵심 포인트

- 새 **이벤트 루프 생성** → 코루틴 실행 → 루프 종료
- 프로그램에서 **딱 한 번만** 호출
- 이미 루프가 실행 중이면 **RuntimeError**



실행의 진입점 : `asyncio.run()`



비활성 설계도

비동기 함수를 정의하는
것과 실행하는 것은
별개

일반 함수처럼 단순히
호출(`main()`)하면 코루틴 객체만
반환될 뿐 실행되지 않음

`asyncio.run()`은 비동기 환경을
생성하고 첫 작업을 구동하는
'엔진의 시동 키' 역할



코드로 보는 비동기 실행의 진입점

```
1 import asyncio
2
3 async def say_hello():
4     print('Hello')
5     await asyncio.sleep(1)
6     print('World')
7
8 async def main():
9     await say_hello()
10
11 asyncio.run(main())
```

Coroutine 생성

비동기적으로 동작할 수 있는 함수 정의

제어권 양보

1초를 기다리는 동안 프로그램이 멈추지 않고 다른 작업을 할 수 있게 함

Event Loop 시작 & 실행

환경을 켜고 main()을 던져 넣음



Awaitable 객체의 핵심

1. 코루틴 (Coroutine)

`async def` 함수를 호출하면 생성되는 객체
스스로 실행될 수 없는 잠재적 작업



2. 태스크 (Task)

`create_task()`로 코루틴을 루프에 등록한 객체
호출 즉시 비동기 실행이 시작됨



참고: 퓨처 (Future)

내부 구현에 사용되는 저수준 객체
직접 다룰 일은 거의 없음



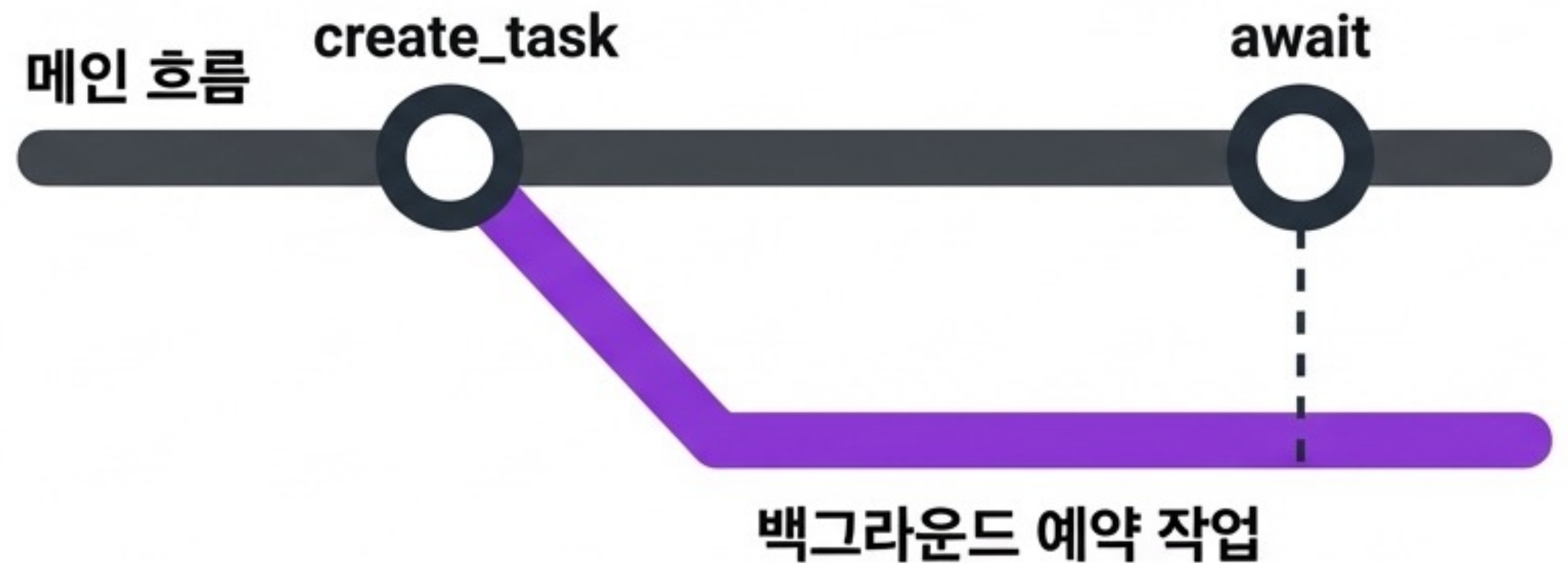
즉시 백그라운드 실행 예약(create_task)

기능

코루틴을 '태스크 객체'로 감싸서 이벤트 루프에 즉시 실행을 예약합니다.

차별점

명시적으로 **await**를 호출하여 기다리지 않아도, 루프에 여유가 생기면 백그라운드에서 이미 작업이 시스템으로 진행됩니다.





태스크의 생성: `asyncio.create_task()`

```
import asyncio

async def fetch_data(url):
    await asyncio.sleep(1)
    # 비동기 대기: 루프를 막지 않고 양보
    return f"{url} done"

async def main():
    # 코루틴을 Task로 등록 -> 백그라운드에서 실행 시작
    task = asyncio.create_task(
        fetch_data("api/1"))
    result = await task # 결과가 필요할 때만 회수
```

`asyncio.create_task(코루틴)`

인자: **코루틴 객체** (코루틴 함수의 호출 결과)

반환값: **Task** 객체 (awaitable)

조건: 실행 중인 **이벤트 루프**가 필요 (async 함수 안에서만)

핵심 포인트

- 코루틴을 Task로 감싸 이벤트 루프에 등록
- `await task` 없이도 백그라운드 실행 (등록 즉시 살아있는 작업이 됨)
- 단, 이벤트 루프가 제어권을 받는 순간(어느 `await` 시점)부터 실제 코드 진행
- 결과가 필요할 때만 `await task`로 회수

create_task와 await의 차이

구분	await 코루틴()	create_task(코루틴())
실행 시점	await를 만나야 실행 시작	create_task 호출 즉시 등록 (루프에 여유 생기면 실행)
실행 방식	순차적 — 하나 끝나야 다음	동시 실행 — 백그라운드에서 진행
결과 받기	await가 곧 결과 반환	await task로 나중에 결과 수집
제어권	완료까지 현재 코루틴 대기	등록 후 바로 다음 코드 실행
사용 예	단일 작업 순서대로 처리	여러 작업을 동시에 처리할 때



Python에서의 비동기 프로그래밍

```
import asyncio

async def task_a():
    print("A: start")
    await asyncio.sleep(1) # 여기서 제어권 양보
    print("A: end")

async def task_b():
    print("B: start")
    await asyncio.sleep(2)
    print("B: end")

async def main():
    await asyncio.gather(task_a(), task_b())

asyncio.run(main())
```

Step 1

A: start 출력 후 **await**에서 제어권 양보

Step 2

B: start 출력 후 **await**에서 대기

Step 3

A: end → B: end 순으로 완료